

INDEX

NAME K.D.D. Sai AbhisamSUBJECT Deep learning lab work

STD. _____ DIV. _____ ROLL NO. _____ SCHOOL _____

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1		Analyze and exploring the deep learning Platforms	24-07-25	He
2		Implement a classifier using open-source data set	7/8/25	He 11/8/25
3		Study of the classifiers with respect to statistical parameters	7/8/25	He 11/8/25
4		Build a simple feed forward neural network to recognize hand written characters	14/8/25	He 14/8/25
5		Study of activation function and its role	28-8-25	He
6		Implement gradient descent & back propagation in deep neural network	09-9-25	He 11/9/25
7		Build a CNN model to classify cats & dogs images	16-9-25	He 11/23/25
8		Experiment using LSTM		He 11/17/25
9		Build a RNN		
10		Perform Compression on MNIST data set using Autoencoder		He 11/17/25
11		to implement experiment using Variational Autoencoder		
12		Implement DCGAN to generate Colour images	27/10	He 11/2/26
13		Understanding the architecture of Pre-trained model	27/10	
14		Pre-trained CNN model as feature extractor using TLM		
15		Implement a YOLO model to detect objects		

samples ~~He~~

9. Build a Recurrent neural network

Aim: to implement and train a Recurrent neural network (RNN) using Pytorch on a real world test dataset

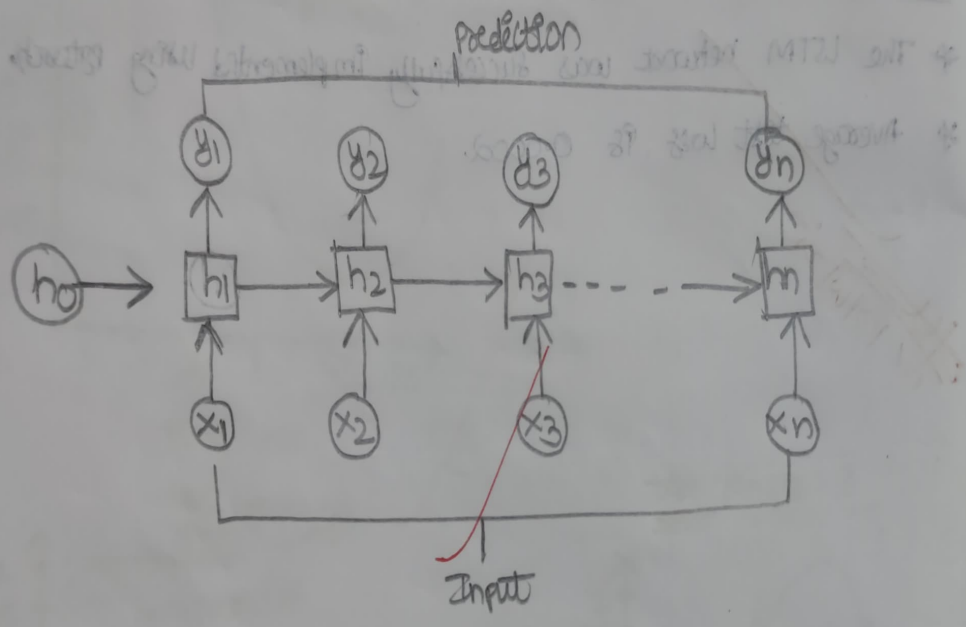
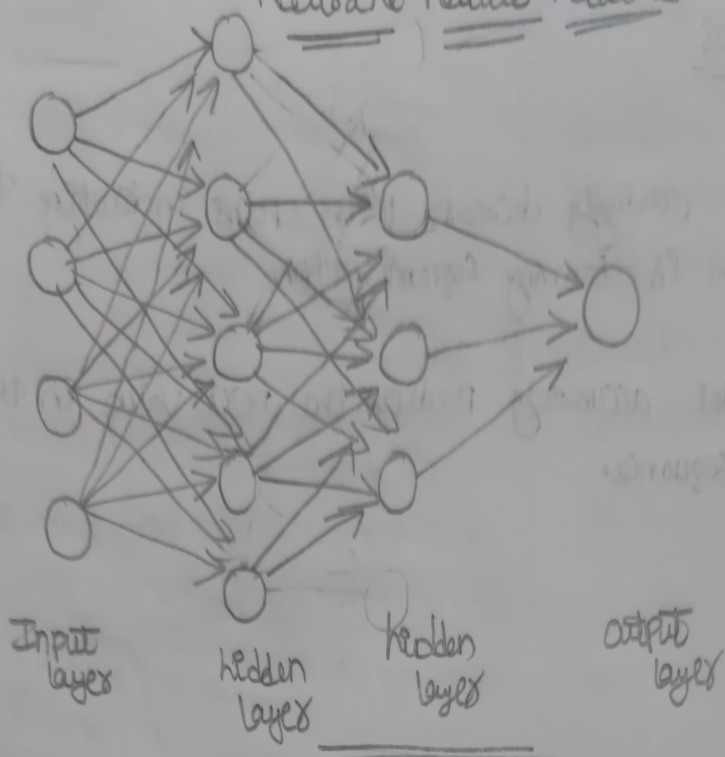
Objective:

- ① to understand the structure and working of RNN's.
- ② to preprocess and tokenize test data for training NN.
- ③ to train RNN's model
- ④ to evaluate model Performance based on binary loss and accuracy

Pseudo Code

- ① Import libraries
- ② Use open data set (or) dataset to load
- ③ Preprocess the data
 - * Convert to tensor batches
- ④ Define RNN's model class.
 - * Embedding layer
 - * RNN layer
 - * fully Connected output layer.
- ⑤ Loss function
- ⑥ Optimizer
- ⑦ train model for given epoch's
 - * zero gradient
 - * forward pass
 - * compute loss
 - * Backward pass

Recurrent neural network



```
[1] import matplotlib.pyplot as plt

# We need to store the loss values during training.
# Let's go back and modify the training loop to store losses.

# (Assuming you would re-run the training cell after this modification)

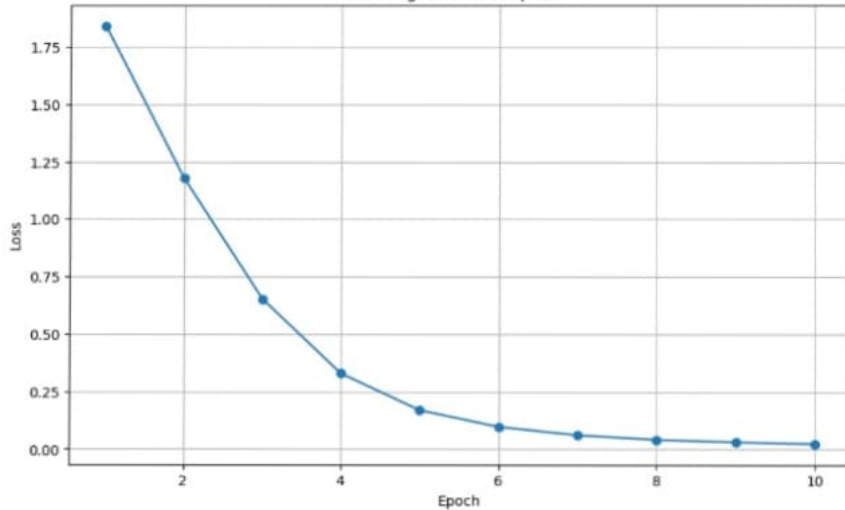
# For now, let's create some dummy loss data for visualization purposes
# In a real scenario, replace this with the actual list of training losses

# Dummy loss data (replace with actual loss data from training)
training_losses = [1.8421, 1.1792, 0.6531, 0.3294, 0.1700, 0.0977, 0.0617, 0.0422, 0.0314, 0.0248]
epochs = range(1, len(training_losses) + 1)

plt.figure(figsize=(10, 6))
plt.plot(epochs, training_losses, marker='o')
plt.title('Training Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)
plt.show()
```

(4)

Training Loss Over Epochs



Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshots folder.

```

# Import the necessary libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Load the dataset
train_data, test_data = tf.keras.datasets.mnist.load_data()

# Preprocess the data
train_data = train_data / 255.0
test_data = test_data / 255.0

# Define the model
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(train_data, train_data[0, :], epochs=10,
        validation_data=(test_data, test_data[0, :]),
        verbose=1)

# Evaluate the model
test_loss, test_acc = model.evaluate(test_data, test_data[0, :])
print(f'Test Loss: {test_loss}, Test Accuracy: {test_acc}')
  
```

Output :

Data preprocessing complete

Vocabulary size : 21

Padded inputs shape : torch.Size([3, 23])

target shape : torch.Size([3, 23])

Epoch [10/100]; loss: 1.8230

Epoch [20/100]; loss: 1.1260

Epoch [30/100]; loss: 0.6053

Epoch [40/100]; loss: 0.3001

Epoch [50/100]; loss: 0.1533

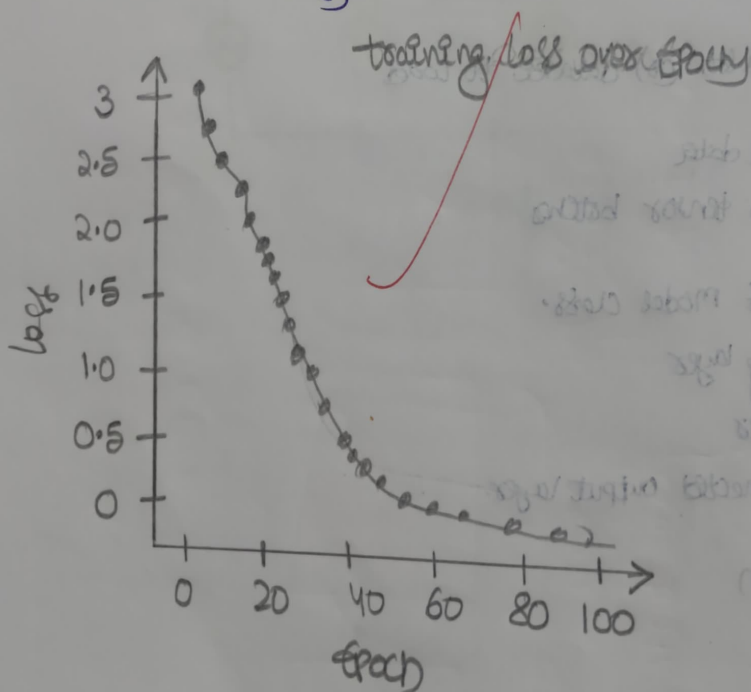
Epoch [60/100]; loss: 0.0877

Epoch [70/100]; loss: 0.0560

Epoch [80/100]; loss: 0.0392

Epoch [90/100]; loss: 0.0296

Epoch [100/100]; loss: 0.0237



* update weights

⑧ evaluate model on test data.

⑨ display accuracy and loss results.

Observations :

- * the loss decreased over epochs, indicating learning progress
- * the accuracy stabilized which is reasonable for a simple RNN without advanced regularization

Result :

The recurrent Neural network was successfully implemented using Pytorch. accuracy on data test, demonstrating its ability to classify as positive or negative.

loss :- 0.0232

accuracy :- 1.0000